

MANUAL TÉCNICO

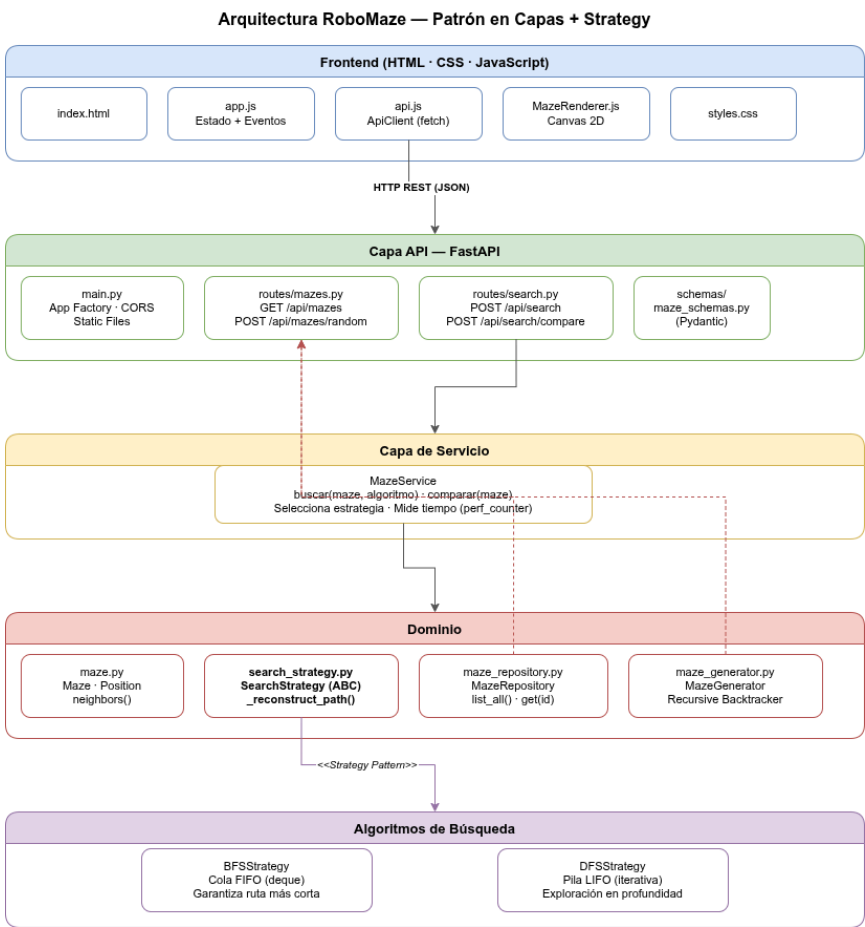
RoboMaze

1. Descripción General

RoboMaze es un sistema web que permite representar laberintos bidimensionales y resolver la búsqueda de rutas entre un punto de inicio y un punto destino mediante los algoritmos BFS y DFS. El backend está desarrollado en Python con FastAPI; el frontend en HTML, CSS y JavaScript puro sin dependencias externas.

2. Arquitectura del Sistema

El backend sigue una arquitectura en capas (N-Tier) con el patrón Strategy para los algoritmos de búsqueda y el patrón Repository para los laberintos predefinidos.



2.1 Capas del Backend

Capa	Responsabilidad	Módulos principales
API	Recibir peticiones HTTP, validar datos de entrada/salida con Pydantic, delegar al servicio	app/api/routes/mazes.py, app/api/routes/search.py
Servicio	Orquestrar la ejecución del algoritmo, medir tiempo, construir el resultado	app/services/maze_service.py
Dominio	Modelar el laberinto, las posiciones y la interfaz de estrategias	app/domain/maze.py, app/domain/search_strategy.py
Algoritmos	Implementar BFS y DFS como estrategias concretas	app/algorithms/bfs.py, app/algorithms/dfs.py
Repositorio	Almacenar y servir los laberintos predefinidos en memoria	app/domain/maze_repository.py
Schemas	Definir los contratos de request/response (Pydantic)	app/schemas/maze_schemas.py

2.2 Patrón Strategy

SearchStrategy es una clase abstracta (ABC) que define el contrato search(maze) → SearchResult. BFSStrategy y DFSStrategy la implementan sin que la capa de servicio necesite conocer sus detalles internos. Esto permite agregar nuevos algoritmos (A*, Dijkstra) sin modificar MazeService.

SearchStrategy (ABC)

├── BFSStrategy → algorithms/bfs.py
└── DFSStrategy → algorithms/dfs.py

2.3 Flujo de una Petición de Búsqueda

Frontend (api.js)

| POST /api/search {grid, start, goal, algoritmo}



routes/search.py ← valida con Pydantic, construye Maze

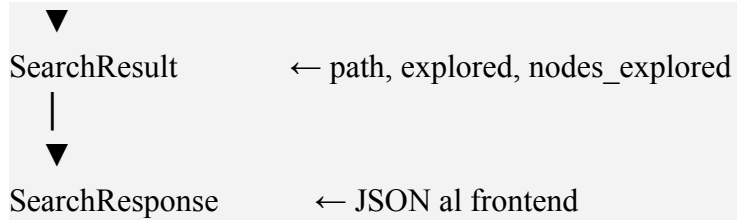


MazeService.buscar() ← selecciona estrategia, cronometra

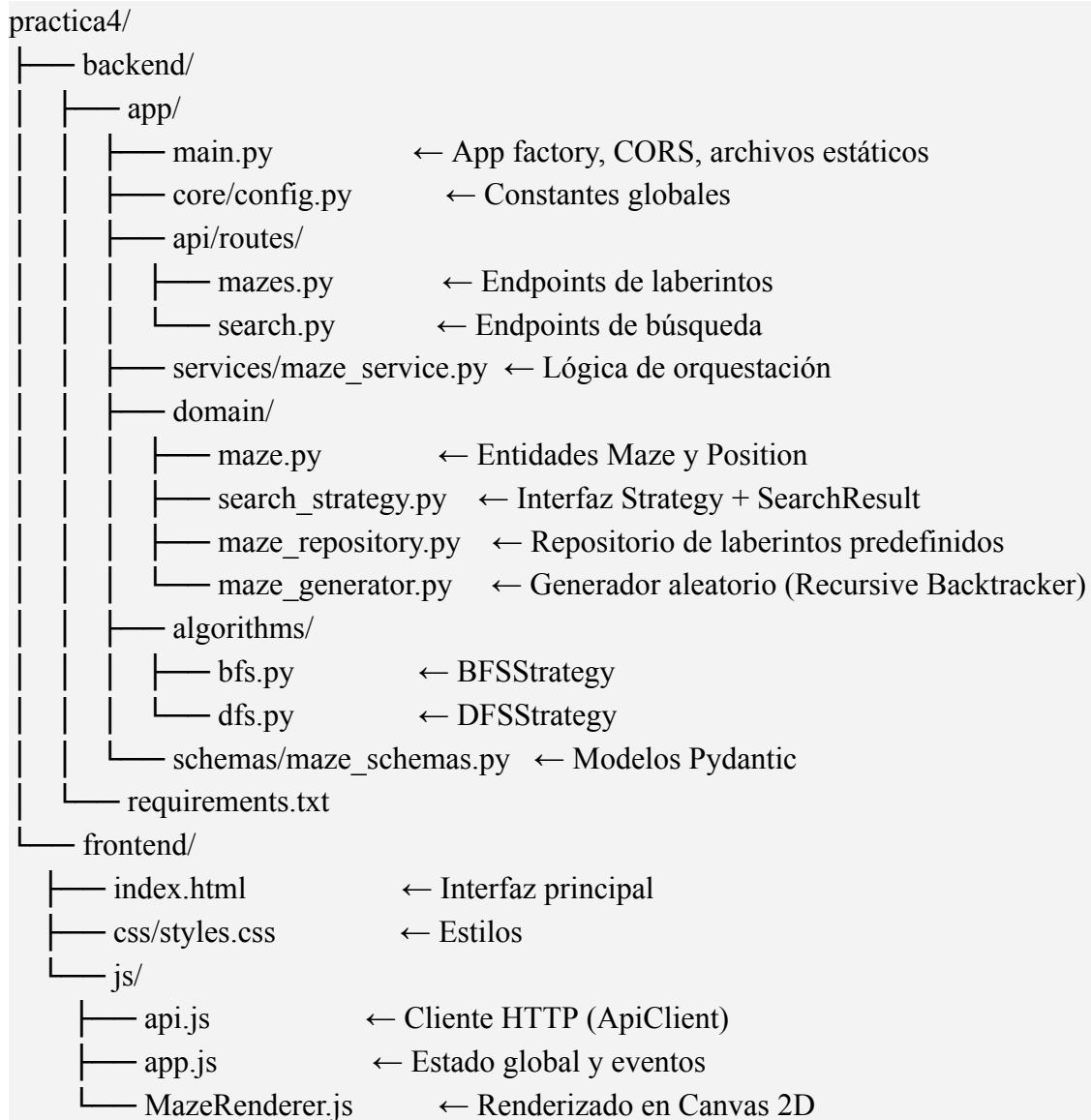


BFSStrategy / DFSStrategy ← ejecuta el algoritmo sobre Maze





3. Estructura del Proyecto



4. Algoritmos de Búsqueda

4.1 BFS — Breadth-First Search

Archivo: backend/app/algorithms/bfs.py

Estructura de datos: Cola FIFO (collections.deque)

Garantía: Encuentra siempre el camino más corto en grafos no ponderados.

Proceso:

- Encolar el nodo inicio; marcarlo como visitado.
- Desencolar el nodo actual; agregarlo a explored.
- Si es la meta → reconstruir el camino y retornar.
- Encolar todos los vecinos transitables no visitados, registrando su predecesor.
- Si la cola se vacía sin encontrar la meta → retornar path=None.

Complejidad:

	Tiempo	Espacio
BFS	$O(V + E)$	$O(V)$

Donde V = celdas transitables, E = conexiones entre ellas.

4.2 DFS — Depth-First Search

Archivo: backend/app/algorithms/dfs.py

Estructura de datos: Pila LIFO explícita (lista Python). Implementación iterativa para evitar el límite de recursión en laberintos grandes.

Garantía: No garantiza el camino más corto. Generalmente explora menos nodos que BFS en laberintos con pocos ramales.

Proceso:

- Apilar (inicio, None).
- Desapilar (actual, padre). Si ya fue visitado, ignorar.
- Marcar como visitado; registrar padre; agregar a explored.
- Si es la meta → reconstruir el camino y retornar.
- Apilar vecinos en orden inverso para mantener el orden natural de exploración.
- Si la pila se vacía → retornar path=None.

Complejidad:

	Tiempo	Espacio
DFS	$O(V + E)$	$O(V)$

4.3 Reconstrucción del Camino

Ambos algoritmos comparten el método estático `_reconstruct_path(parent, goal)` heredado de `SearchStrategy`. Recorre el diccionario de padres desde la meta hacia el inicio y revierte la lista para obtener el camino en orden correcto.

5. API REST

URL base: http://localhost:8000/api

Documentación interactiva (Swagger UI): http://localhost:8000/docs

5.1 Laberintos

GET /api/mazes

Lista los metadatos de los laberintos predefinidos.

Respuesta:

```
[{"id": "facil", "nombre": "...", "dificultad": "Fácil", "rows": 13, "cols": 13 }, ...]
```

GET /api/mazes/{maze_id}

Devuelve el laberinto predefinido completo (grid + start + goal). Error 404 si el maze_id no existe.

POST /api/mazes/random

Genera un laberinto perfecto aleatorio usando el algoritmo Recursive Backtracker.

Body: { "cols": 15, "rows": 15 } | Restricciones: cols y rows entre 5 y 30.

5.2 Búsqueda

POST /api/search

Ejecuta BFS o DFS sobre el laberinto enviado.

```
{ "grid": [...], "start": {"row": 1, "col": 1}, "goal": {"row": 11, "col": 11}, "algoritmo": "BFS" }
```

POST /api/search/compare

Ejecuta BFS y DFS sobre el mismo laberinto en una sola llamada.

```
{ "bfs": { ... }, "dfs": { ... } }
```

6. Requerimientos Funcionales

ID	Requerimiento
RF-01	El sistema representa el laberinto como una cuadrícula bidimensional de celdas (0 = abierta, 1 = muro).
RF-02	El usuario puede cargar uno de los 5 laberintos predefinidos con distintos niveles de dificultad.
RF-03	El usuario puede generar un laberinto aleatorio especificando ancho y alto (5–30 celdas).
RF-04	El usuario puede definir manualmente la posición de inicio haciendo clic en el canvas.
RF-05	El usuario puede definir manualmente la posición de meta haciendo clic en el canvas.

ID	Requerimiento
RF-06	El usuario puede colocar y eliminar muros haciendo clic o arrastrando sobre el canvas.
RF-07	El sistema ejecuta el algoritmo BFS de forma independiente y muestra la ruta encontrada.
RF-08	El sistema ejecuta el algoritmo DFS de forma independiente y muestra la ruta encontrada.
RF-09	El sistema muestra gráficamente el recorrido de celdas exploradas durante la búsqueda.
RF-10	El sistema muestra la cantidad de nodos explorados por cada algoritmo.
RF-11	El sistema muestra el tiempo de ejecución de cada algoritmo en milisegundos.
RF-12	El sistema permite comparar BFS y DFS en una sola operación, mostrando métricas de ambos.
RF-13	El sistema informa al usuario cuando no existe ruta válida entre inicio y meta.
RF-14	Al mover el inicio o la meta, los resultados de búsqueda anteriores se eliminan del canvas.

7. Requerimientos No Funcionales

ID	Requerimiento	Categoría
RNF-01	Los algoritmos BFS y DFS deben responder en menos de 500 ms para laberintos de hasta 31×31 celdas.	Rendimiento
RNF-02	El backend no persiste ningún estado entre peticiones; es completamente stateless.	Escalabilidad
RNF-03	El frontend no requiere instalación ni build steps: funciona sirviendo archivos estáticos desde FastAPI.	Usabilidad
RNF-04	La interfaz es responsiva y el canvas se adapta al tamaño del contenedor al redimensionar la ventana.	Usabilidad
RNF-05	La arquitectura en capas separa API, servicio, dominio y algoritmos; agregar un nuevo algoritmo requiere solo implementar SearchStrategy sin modificar otras capas.	Mantenibilidad

ID	Requerimiento	Categoría
RNF-06	Todos los esquemas de entrada y salida de la API están validados con Pydantic, rechazando datos inválidos con HTTP 400.	Confiabilidad
RNF-07	El backend está desarrollado exclusivamente en Python 3.x; sin uso de bases de datos ni servicios externos.	Restricción técnica
RNF-08	Los algoritmos de búsqueda no utilizan librerías externas para la resolución de rutas; están implementados manualmente.	Restricción académica

8. Posibles Mejoras Futuras

- **Algoritmo A*:** Agregar AStarStrategy con heurística Manhattan para encontrar rutas óptimas de forma más eficiente que BFS en laberintos grandes.
- **Animación de exploración:** Hacer que las celdas exploradas aparezcan una a una con un retardo configurable.
- **Exportación de resultados:** Permitir descargar las métricas de comparación en CSV o el laberinto como imagen PNG.
- **Múltiples metas:** Extender Maze para soportar más de un nodo objetivo y adaptar los algoritmos para encontrar la meta más cercana.
- **Obstáculos dinámicos:** Permitir agregar y eliminar muros mientras el algoritmo está en ejecución.
- **Persistencia de laberintos:** Guardar laberintos personalizados en localStorage del navegador o en un archivo JSON en el servidor.